



International University of Sarajevo

Pharmacy Database

Database Management Project

CS306

Esma Hodžić 240201006

Ajša Crnić 240302004

Motivational statement

About the business:

The Pharmacy Management System is an extensive database design that efficiently manages the core operations of a pharmacy. The design focuses on the arrangement of important data about medicines, suppliers, buyers, and transactions in order to maintain it safely. In addition, the system assists the routine functioning of a pharmacy through the tracking of medicine inventory, customers' orders, and updating information about suppliers. Through the well-arranged tables and their connections, the system ensures that all information is consistent and increases efficiency in the daily activities of the pharmacy.

Problem statement:

Currently, the operation of the pharmacy involves a lot of manual work regarding record keeping as well as the use of disparate paper documents to organize inventory management, contacts with suppliers, and history of sales. Not only does the use of such a method of operation create inefficiency, but there is also the issue of a possibility of making critical mistakes related to that approach. In particular, it is often challenging for employees to establish the exact time when certain medications expire or to find out whether or not they are already out of stock. That problem used to cause both lost sales opportunities and increased likelihood of dispensing potentially expired products. Additionally, due to the absence of a centralized and relational database in this case, it is difficult for employees to find relevant data in order to tie a sale with a particular customer or to identify which supplier should be contacted to obtain the required medication. The use of a relational MySQL database will help to improve the situation by introducing efficiency into the process of linking data to avoid mistakes. The primary information reports required from this database are a “stock availability report” and an “expiring products report” to ensure we never compromise on patient safety.

Business Rules

Based on the problem domain of pharmaceutical data management, the following business rules have been established for the database:

- Each medicine must have a unique identifier and must be associated with one supplier.
- A supplier can provide multiple medicines, but each medicine is supplied by only one supplier.

- Each customer can make multiple purchases, but each sale is associated with only one customer.
- The quantity of a medicine must always be a non-negative value.
- Medicines must have valid expiration dates, and expired medicines should not be available for sale.
- Prices of medicines must be a positive value
- A supplier or medicine cannot be removed from the database if it is still being used in other tables, such as sales, to avoid creating missing or invalid data.

Relational Model

The design of the Pharmacy Management System is represented through an Entity-Relationship Diagram (ERD) that models the real-world interactions within a pharmacy. This design ensures that all business rules are enforced, and that data is stored logically to prevent redundancy.

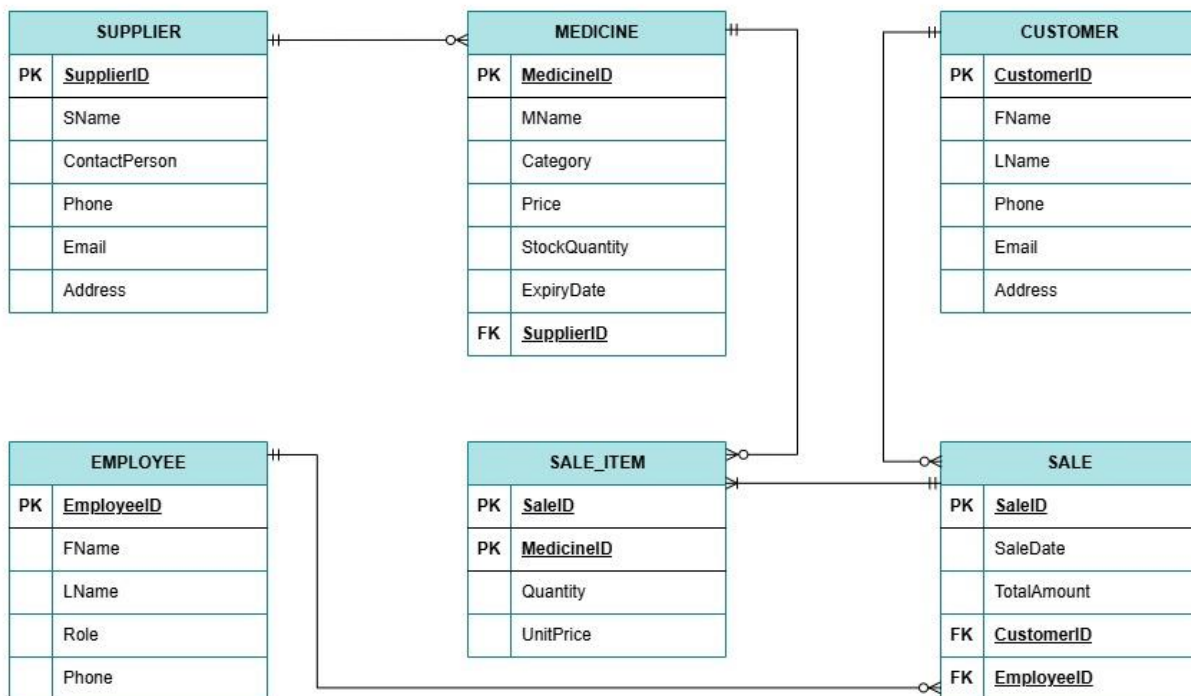


Figure 1. Entity Relationship Diagram

The following tables and columns define the schema of the database:

- **SUPPLIER:** Stores information about the companies providing the medication.
 - Attributes: SupplierID (PK), SName, ContactPerson, Phone, Email, Address.
- **MEDICINE:** Contains the details of the products available for sale.
 - Attributes: MedicineID (PK), MName, Category, Price, StockQuantity, ExpiryDate, SupplierID (FK).
- **CUSTOMER:** Maintains records of individuals purchasing items.
 - Attributes: CustomerID (PK), FName, LName, Phone, Email, Address.
- **EMPLOYEE:** Identifies the staff members processing the transactions.
 - Attributes: EmployeeID (PK), FName, LName, Role, Phone.
- **SALE:** Records the header information for every transaction.
 - Attributes: SaleID (PK), SaleDate, TotalAmount, CustomerID (FK), EmployeeID (FK).
- **SALE_ITEM:** A bridge table that handles the many-to-many relationship between Sales and Medicines.
 - Attributes: [SaleID, MedicineID] (Composite PK), Quantity, UnitPrice.

These entities are connected using Crow's Foot notation to define the cardinality of each relationship. For example CUSTOMER and SALE have a (1,1) : (0,N) cardinality, meaning that a sale has to be linked to a customer, but a customer can exist in a database without having made a purchase yet (0), or they can have many past sales (N).

The connectivity between these entities shows how data flows through the system:

SUPPLIER to MEDICINE (1:M), as one supplier provides many medicines, whereas each medicine comes from one supplier.

CUSTOMER to SALE (1:M), as one customer can have many sales records, but each sale belongs to a single customer.

EMPLOYEE to SALE (1:M), as one employee can process many sales, but each sale is recorded by one specific employee.

And finally, **SALE to MEDICINE (M:N)**, as a single sale can include many medicines, and a specific medicine can appear in many different sales. This is resolved through the SALE_ITEM table.

The relationships can be justified using semantic nets:

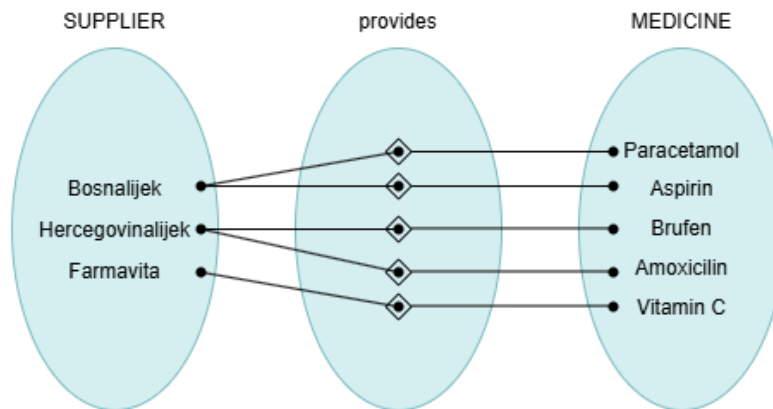


Figure 2. Semantic net for SUPPLIER and MEDICINE

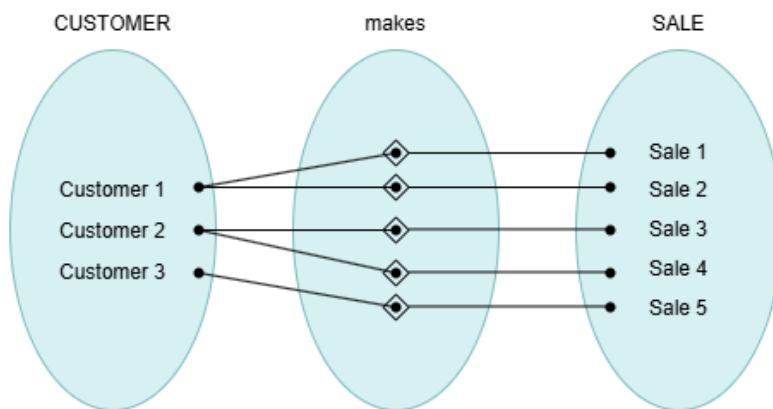


Figure 3. Semantic net for CUSTOMER and SALE

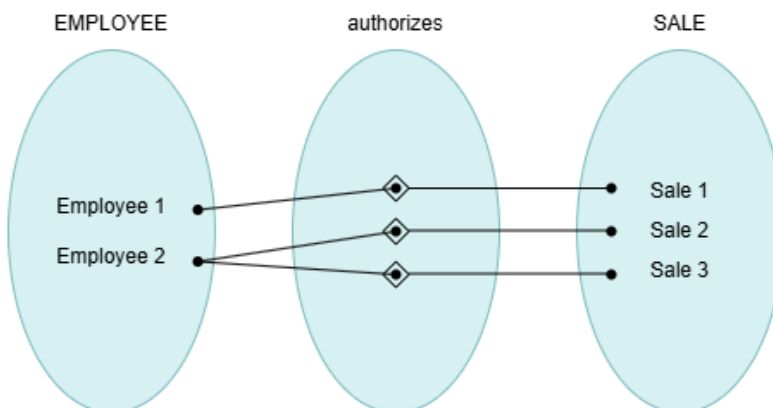


Figure 4. Semantic net for EMPLOYEE and SALE

In designing the Pharmacy Management System, the model is developed in compliance with 3NF rules. The objective here is to maintain data consistency and avoid any forms of redundancy. In order to achieve the 2NF standard, the model will ensure that all non-key attributes are entirely dependent on the primary key. A significant consideration of this model is the use of the SALE_ITEM table, which employs the composite key SaleID and MedicineID. The use of this composite key will be instrumental in facilitating the many-to-many relationship between sales and products to avoid any form of duplicate data. This model further satisfies the requirements of 3NF through the elimination of transitive dependencies. For example, data such as the supplier contact details or the medicine category are captured in separate tables without appearing in the sales entry. In essence, each piece of data will have one occurrence only. The significance of this strategy will enable the avoidance of data anomalies when inserting, modifying, or deleting entries in the database.

Information reports

The **six** tables were additionally added into SQL based on the entities, attributes, and relationships visualized in the graph. These tables are called **MEDICINE**, **EMPLOYEE**, **CUSTOMER**, **SALE**, **SUPPLIER**, and **SALE_ITEM**. Values were also added into each table using SQL commands for creating tables and inserting data.

These tables now represent the values for each type of entity, such as customers, employees, available medicines, suppliers, sales, and sale details.

Below are some examples of code used for creating some of the tables:

```
CREATE TABLE MEDICINE(  
    MedicineID INTEGER PRIMARY KEY auto_increment,  
    MName VARCHAR(50) NOT NULL,  
    Category VARCHAR(50) NOT NULL,  
    Price DECIMAL(8,2) NOT NULL CHECK (Price>0),  
    StockQuantity INT NOT NULL CHECK (StockQuantity >= 0),  
    ExpiryDate DATE NOT NULL,  
    SupplierID INT NOT NULL,  
    foreign key (SupplierID) REFERENCES SUPPLIER(SupplierID));  
  
INSERT INTO MEDICINE(MedicineID, MName, Category, Price, StockQuantity, ExpiryDate, SupplierID)  
VALUES  
(201, 'Paracetamol', 'Painkiller', 3.50, 120, '2027-05-15', 101),  
(202, 'Brufen', 'Anti-inflammatory', 6.20, 80, '2026-11-20', 102),  
(203, 'Aspirin', 'Painkiller', 4.00, 100, '2027-01-10', 101),  
(204, 'Vitamin C', 'Supplement', 5.50, 150, '2028-03-25', 103),  
(205, 'Amoxicillin', 'Antibiotic', 12.00, 60, '2026-09-30', 102);
```

Figure 5. Creating table MEDICINE

```

CREATE TABLE CUSTOMER(
  CustomerID INT PRIMARY KEY auto_increment,
  FName VARCHAR(50) NOT NULL,
  LName VARCHAR(50) NOT NULL,
  Phone VARCHAR(20) NOT NULL,
  Email VARCHAR(50) UNIQUE,
  Adress VARCHAR(100)
);

INSERT INTO CUSTOMER (CustomerID, FName, LName, Phone, Email, Adress)
VALUES
(301, 'Amna', 'Hodžić', '+387 61 111 222', 'amna@gmail.com', 'Sarajevo'),
(302, 'Emina', 'Karić', '+387 62 333 444', 'emina@gmail.com', 'Zenica'),
(303, 'Adnan', 'Begić', '+387 63 555 666', 'adnan@gmail.com', 'Mostar');

```

Figure 6. Creating table CUSTOMER

```

CREATE TABLE EMPLOYEE(
  EmployeeID INTEGER PRIMARY KEY AUTO_INCREMENT,
  FName VARCHAR(50),
  LName VARCHAR(50),
  Role VARCHAR(50),
  Phone VARCHAR(20)
);

INSERT INTO EMPLOYEE (EmployeeID, FName, LName, Role, Phone)
VALUES
(401, 'Lejla', 'Softić', 'Pharmacist', '+387 61 222 333'),
(402, 'Emir', 'Alić', 'Cashier', '+387 62 444 555'),
(403, 'Sara', 'Delić', 'Manager', '+387 63 666 777');

```

Figure 7. Creating table EMPLOYEE

This demonstrates the principle of how tables representing each entity with its attributes are implemented and how each table contains corresponding values.

In [Metabase](#), the tables were also created, providing a clear example of how a pharmacy database should be structured. This not only gives a preview of all the data a pharmacy system contains, but also shows how the data is distributed and how important information can be analyzed from the database. Furthermore, it represents real-world pharmacy operations and the decision-making process. .

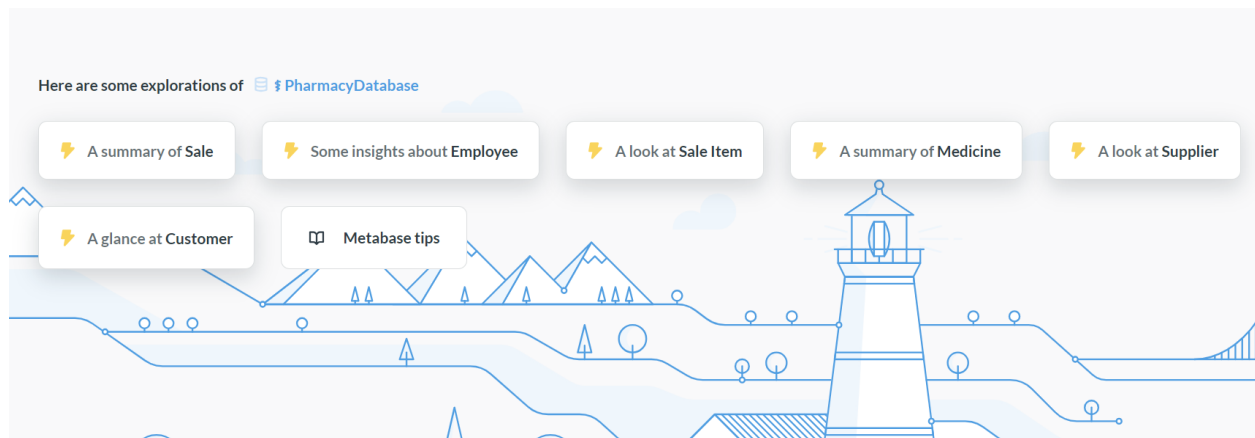


Figure 8. Metabase of Pharmacy Database

After examining each of the exploration buttons in [Metabase](#), different analytical views and summaries of the Pharmacy Database were obtained.

In the medicine section of the pharmacy database, there are different types of distributed information, such as medicine stock quantity over time, timestamps of medicine sales by day of the week, and organization of medicines by price or supplier. These analyses are very important for managing and tracking data related to medicines in a pharmacy.

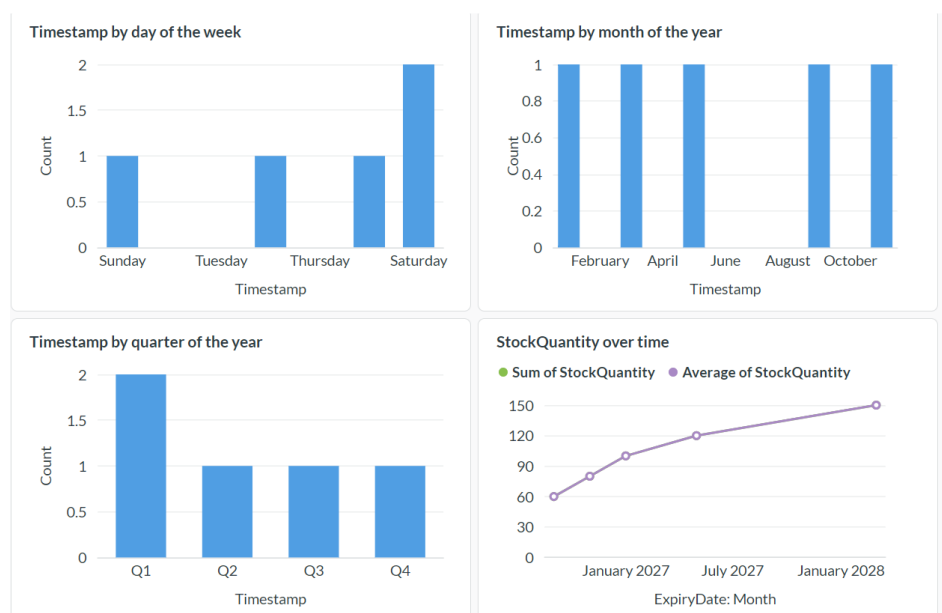


Figure 9. Medicine details

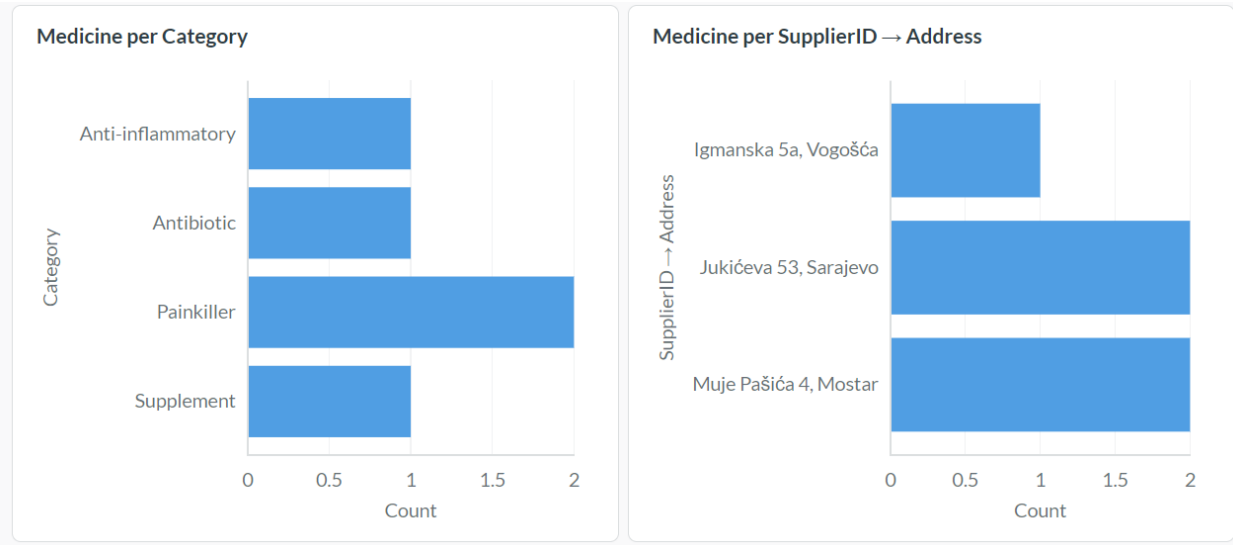


Figure 10. Medicine details



Figure 11. Medicine ordering

For each type of data, different variations of information are also distributed and analyzed.

Some important questions were created based on the data from this pharmacy database, and the obtained details were used to analyze and represent the data more effectively by answering those questions. Some of the answers represented important information such as: every medicine with its supplier, each sale with the customer who bought the medicine and the employee who completed the sale, details of each sold medicine, the price of each medicine, the total number of medicines in stock, the most expensive medicine, medicines that expire before 1 January 2027, the number of medicines belonging to each category, medicines ordered by price from highest to lowest, medicines that are more expensive than the average price, customers who made at least one purchase, and each sale connected with its customer and employee.

These are some of the very important pieces of information that this database stores and that can be represented and analyzed in programs such as SQL using queries.

```
-- Each sale with customer who bought it and employee who made sale
SELECT SA.SaleID, SA.SaleDate, C.FName AS CustomerName, C.LName AS CustomerSurname,
       E.FName AS EmployeeName, E.LName AS EmployeeSurname, SA.TotalAmount
FROM SALE AS SA
JOIN CUSTOMER AS C ON SA.CustomerID = C.CustomerID
JOIN EMPLOYEE E ON SA.EmployeeID = E.EmployeeID;

-- Shows details of each sold medicine, how much each of them cost
SELECT SA.SaleID, SA.SaleDate, M.MName, SI.Quantity, SI.UnitPrice,
       SI.Quantity * SI.UnitPrice AS ItemTotal
FROM SALE_ITEM AS SI
JOIN SALE AS SA ON SI.SaleID = SA.SaleID
JOIN MEDICINE M ON SI.MedicineID = M.MedicineID;
```

Figure 12. queries

Figure 8 represents each sale together with the customer who bought the medicine, as well as the employee who completed the sale. The second query represents details such as the cost of a specific medicine.

The tables visually represented in [Metabase](#) look as follows:

SaleID	SaleDate	MName	Quantity	UnitPrice	ItemTotal	+
501	May 1, 2026	Paracetamol	2	3.5	7	
501	May 1, 2026	Aspirin	1	4	4	
502	May 2, 2026	Brufen	1	6.2	6.2	
502	May 2, 2026	Vitamin C	2	5.5	11	
503	May 3, 2026	Amoxicillin	2	12.8	25.6	

Figure 13. Details on medicine

SaleID	SaleDate	CustomerName	CustomerSurname	EmployeeName	EmployeeSurname	TotalAmount	+
501	May 1, 2026	Amna	Hodžić	Lejla	Softić	10	
502	May 2, 2026	Emina	Karić	Emir	Alić	18.3	
503	May 3, 2026	Adnan	Begić	Lejla	Softić	25.6	

Figure 14. Customer who bought it and employee who sold it

```
-- Shows the most expensive medicine
SELECT MName, Price
FROM MEDICINE
ORDER BY Price DESC
LIMIT 1;

-- Medicines that expire before 1 January 2027
SELECT *
FROM MEDICINE
WHERE ExpiryDate < '2027-01-01';

-- Shows how many medicines belong to each category
SELECT Category, COUNT(*) AS NumberOfMedicines
FROM MEDICINE
GROUP BY Category;
```

Figure 15. queries

These queries represent the most expensive medicine, as well as medicine that expires before 1.1.2027. , also represents table of medicines belonging to each category.

MedicineID	MName	Category	Price	StockQuantity	ExpiryDate	SupplierID	+
202	Brufen	Anti-inflammatory	6.2	80	November 20, 2026	102	
205	Amoxicillin	Antibiotic	12.8	60	September 30, 2026	102	

Figure 16. Medicine whose expiring date before 1.1.2027.

MName	Price	+
Amoxicillin	12.8	

Figure 17. Most expensive medicine

Category	NumberOfMedicines	+
Painkiller	2	
Anti-inflammatory	1	
Supplement	1	
Antibiotic	1	

Figure 18. Number of Medicines per category

The following queries represent medicines ordered by price, medicines that are more expensive than the average price, and customers who made at least one purchase:

```

-- Shows medicines ordered by price from highest to lowest
SELECT MName, Category, Price
FROM MEDICINE
ORDER BY Price DESC;

-- Shows medicine that are more expensive than average price
SELECT MName, Price
FROM MEDICINE
WHERE Price > (
    SELECT avg(Price) FROM MEDICINE
);

-- Shows customers who made at least 1 sale:
SELECT FName, LName
FROM CUSTOMER
WHERE CustomerID IN (
    SELECT CustomerID FROM SALE);

```

Figure 19. queries

And after implementation following tables/ graphs were obtained:

FName	LName	+
Amna	Hodžić	
Emina	Karić	
Adnan	Begić	

Figure 20. Customers with at least 1 sale



Figure 21. Medicine more expensive than average price (Amonexil)

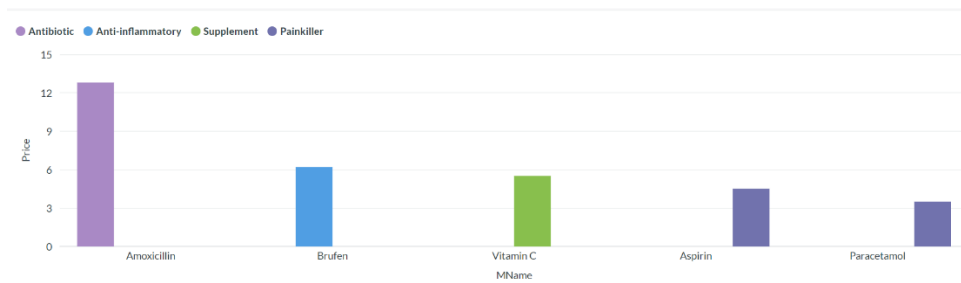


Figure 22. Medicine ordered by price from highest to lowest

Lastly, some of the queries were also:

```
-- Shows sold medicine and total price for each item
SELECT SI.SaleID, M.MName, SI.Quantity, SI.UnitPrice,
(SI.Quantity * SI.UnitPrice) AS ItemTotal
FROM SALE_ITEM AS SI INNER JOIN MEDICINE AS M
ON SI.MedicineID = M.MedicineID;

-- Shows each sale with customer and employee
SELECT SA.SaleID, SA.SaleDate, C.FName AS CustomerName,
E.FName AS EmployeeName, SA.TotalAmount
FROM (SALE AS SA JOIN CUSTOMER AS C
ON SA.CustomerID = C.CustomerID)
JOIN EMPLOYEE AS E ON SA.EmployeeID = E.EmployeeID;
```

Figure 23. Queries

One query shows how many medicines were sold, as well as the total price for each sold item. Another query shows each sale together with the customer and employee involved in the purchase. Both queries use **JOIN** in SQL to gather data from different entities.

Later, the tables created from this information are:

SaleID	MName	Quantity	UnitPrice	ItemTotal	+
501	Paracetamol	2	3.5	7	
501	Aspirin	1	4	4	
502	Brufen	1	6.2	6.2	
502	Vitamin C	2	5.5	11	
503	Amoxicillin	2	12.8	25.6	

Figure 24. Medicine sold with total price

SaleID	SaleDate	CustomerName	EmployeeName	TotalAmount	+
501	May 1, 2026	Amna	Lejla	10	
502	May 2, 2026	Emina	Emir	18.3	
503	May 3, 2026	Adnan	Lejla	25.6	

Figure 25. Each sale with customer and employee

In the process of creating some of this information and tables from the data, certain information was also changed and updated. New data was adapted to the existing data, which was also done in the same program:

```
-- Changes the price of medicine 203 to 4.50
UPDATE MEDICINE
SET Price = 4.50
WHERE MedicineID = 203;
```

Figure 26. Change of price

It is not unusual for items, such as medicines in this case, to change in price. In such situations, after deciding to increase or decrease the price of a medicine for any reason, that information also needs to be updated in the pharmacy database system.

The same applies to any other change in the pharmacy's data.

Additionally, when a medicine is purchased, the quantity in stock of that specific medicine decreases. When the supplier delivers new medicines, the quantity increases. This is also an important aspect that must be considered while creating a database system.

```
-- Decreases the stock of medicine 201 by 2
UPDATE MEDICINE
SET StockQuantity = StockQuantity - 2
WHERE MedicineID = 201;
```

Figure 27. Decreasing in stock of some medicine

Finally, [Metabase](#), together with all the stored data and information, provides the opportunity to visualize and access different types of information through the web. It gives visual representations of questions related to the pharmacy database, such as who the suppliers are or which medicine has the highest price.

The data can also be represented using dashboards, which visualize all important information in one place.

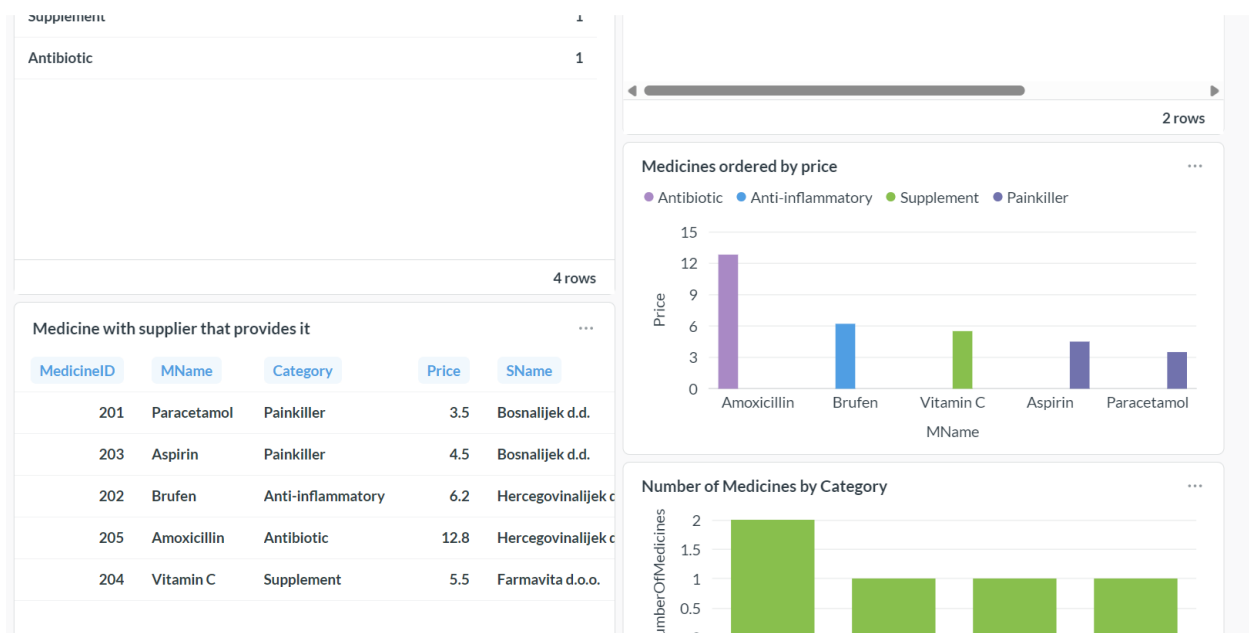


Figure 28. Preview of dashboard

The information has been recorded, the data has been processed, and the database has been represented by this system.

Conclusion

The Pharmacy Management System project managed to revolutionize the internal processes of the pharmacy's core operations, transforming them from being inconsistent and dependent on manual procedures to becoming very efficient and highly functional automated processes. As a result, the database developed for this system is not only reliable, but also capable of performing many tasks by itself, such as helping the pharmacy operate efficiently. Throughout the entire development process, it has been proven that minor operations, such as changing the price of medicines or recording the number of goods that arrived after a certain delivery, could easily be conducted without risking any other information within the database.

As a major achievement of the project, one should mention the implementation of a 3NF structure, which made sure that data redundancy would not become an issue anymore. In particular, the use of composite keys in the SALE_ITEM table allowed us to link together the complex connection between sales and medicine stock. At the same time, the technical capabilities were realized with the help of the Metabase system, in which we have designed a visual dashboard that helped interpret data and make critical conclusions regarding different aspects of the pharmacy's operation, such as which medicines cost the most or which suppliers are the most active.

Finally, in conclusion, one may state that creating the Pharmacy Management System database helped establish the protection of the company's interests as well as those of its customers. Many issues were able to be resolved with the database. For example, with the ability of tracking expiration dates and the quantity of stock available to conduct operations automatically, the pharmacy will be able to prevent the selling of expired products to customers, increasing the overall level of safety in its operations. Through the integration of a professional relational framework with practical experience in manipulating data and generating visual reports, this project provides an end-to-end approach to ensure that any pharmacy runs with precision, safety, and a data-based insight into all its aspects of the inventory and sales records.